

P1018R15

C++ Language Evolution status pandemic edition 2022/01–2022/02

Published Proposal, 2022-02-15

This version:

<http://wg21.link/P1018r15>

Author:

[JF Bastien](#) (Woven Planet)

Audience:

WG21, EWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Source:

github.com/jfbastien/papers/blob/master/source/P1018r15.bs

Abstract

This paper is a collection of items that the C++ Language Evolution group has worked on in the latest meeting, their status, and plans for the future.

Table of Contents

- 1 **Executive summary**
- 2 **Papers of note**
- 3 **Tentatively ready papers**
- 4 **Issue Processing**
- 5 **Polls**
 - 5.1 P2280r3 Using unknown references in constant expressions
 - 5.2 P2468r1 The Equality Operator You Are Looking For
 - 5.3 P2327r1 De-deprecating `volatile` compound operations
 - 5.4 P2266r2 Simpler implicit move
 - 5.5 P1467r8 Extended floating-point types and standard names
 - 5.6 P2350r2 `constexpr class`
 - 5.7 P1169r3 `static operator()`
 - 5.8 P1774r5 Portable assumptions
 - 5.9 P1494r2 Partial program correctness
 - 5.10 P2448r0 Relaxing some `constexpr` restrictions
 - 5.11 P2437r0 Support for `#warning`
 - 5.12 P2324r1 Labels at the end of compound statements (C compatibility)
 - 5.13 P2071r1 Named universal character escapes
 - 5.14 P2295r5 Support for UTF-8 as a portable source file encoding
 - 5.15 P2290r2 Delimited escape sequences
 - 5.16 P2362r3 Remove non-encodable wide character literals and multicharacter wide character literals

6 **Polling Process**

7 **Teleconferences**

8 **Remaining Open Issues**

9 **Near-future EWG plans**

References

Informative References

§ 1. Executive summary

We have not met in-person since the February 2020 meeting in Prague because of the global pandemic. We're instead holding weekly teleconferences, as detailed in [\[P2145R1\]](#). We focus on providing non-final guidance, and will use electronic straw polls as detailed in [\[P2195R0\]](#) to move papers and issues forward in an asynchronous manner.

Our main achievements have been:

- **Issue processing:** most of the 50 language evolution issues have proposed resolutions, and a large number of them have been voted on through electronic polling.
- **C++23:** we've worked on papers for C++23 and later. As of January 2022, EWG is not considering new papers for C++23.
- **Incubation:** we've acted as EWG-I and "incubated" some early papers by providing early feedback to authors.

This paper outlines:

- The work achieved in [§ 7 Teleconferences](#);
- Lists the [§ 5 Polls](#) results for the Early 2022 polling period and explain the [§ 6 Polling Process](#) (see [\[P1018r9\]](#) for the results of the February 2021 polling period, [\[P1018r11\]](#) for the results of the May 2021 polling period, and [\[P1018r13\]](#) for the results of the August polling period);
- Lists the remaining outstanding issues in [§ 8 Remaining Open Issues](#), some are still open while others are waiting for polling;
- [§ 9 Near-future EWG plans](#).

§ 2. Papers of note

- [\[P1000R4\]](#) C++ IS schedule
- [\[P0592R4\]](#) To boldly suggest an overall plan for C++23
- [\[P1999R0\]](#) Process: double-check evolutionary material via a Tentatively Ready status
- [\[P2195R0\]](#) Electronic Straw Polls
- [\[P2145R1\]](#) Evolving C++ Remotely

§ 3. Tentatively ready papers

Following our process in [\[P1999R0\]](#), we usually mark papers as tentatively ready for CWG. We would usually take a brief look at the next meeting, and if nothing particular concerns anyone, send them to CWG. However, given the pandemic, we've decided to provide guidance only in virtual teleconferences, and have an asynchronous polling mechanism to officially send papers to CWG or other groups as detailed in [\[P2195R0\]](#).

You can follow the lists of papers on GitHub:

- [tentatively ready papers](#),

- [EWG vote on me papers.](#)

§ 4. Issue Processing

We've reviewed 50 Language Evolution issues at the Core groups' request, and have tentative resolutions for most. We don't want to poll all of these at the same time, and therefore only poll a subset in each polling period, reserving other issues for later polling periods. We've therefore only polled the "tentatively ready" issues (since they're tied to papers, and polled with said papers, as outlined below), as well as the "resolved" issues since telecon attendees believe that prior work has already addressed the issues.

[§ 8 Remaining Open Issues](#) contains a list of issues which aren't being voted on in this polling period.

§ 5. Polls

The polls which EWG took in the January 2022 polling period, and the poll results, are:

§ 5.1. P2280r3 Using unknown references in constant expressions

- [\[P2280r3\]](#)
- [GitHub issue](#)
- **Highlight:** `constexpr` usage of unknown references doesn't work, that's surprising, make it work as expected.
- **🗳️ Poll:** Forward P2280r3 "Using unknown references in constant expressions" to Core for C++23. Treat it as a Defect Report.

Poll votes:

SF	F	N	A	SA
20	12	2	0	0
Abstain: 11				

Poll outcome:  consensus

Salient comments:

- **Strong Favor:** While this paper de-fact introduce the special-case for the `constexpr` function, I think that it leads to more intuitive and teachable behaviour of `constexpr` functions.
- **Strong Favor:** This covers cases where the compiler knows what I am asking, but refuses to answer because it's not allowed to. This allows the compiler to tell me what it knows well.
- **Strong Favor:** It is pretty hard to explain to a non-language-lawyer why asking for the size of a by-reference `std::array` parameter is non-constant but doing so for a by-value `std::array` parameter is constant; and references + this seems like a good place to draw the line. Ship it.
- **Strong Favor:** I believe this is useful and relatively easily implementable.
- **Favor:** Constant evaluation is unequivocally weird: it can use non-static local variables initialized with constant expressions when no call to the function ever takes place, and it notionally occurs at runtime even though its failure can make the program ill-formed. This paper is in line with these properties, freely inventing objects with overlapping storage of types that can never be constructed and changing lvalues with known referents to refer to such phantoms instead. That said, the change somehow makes the language less surprising, as the frequency of confusion over the non-`static std::array::size` illustrates. As a matter of convenience, then, if not of coherence, it is an improvement.

§ 5.2. P2468r1 The Equality Operator You Are Looking For

- [\[P2468r1\]](#)
- [GitHub issue](#)
- **Highlight:** Make rewriting equality in expressions less of a breaking change.

-  **Poll:** Forward P2468r1 "The Equality Operator You Are Looking For" to Core for C++23. Treat it as a Defect Report against C++20.

Poll votes:

SF	F	N	A	SA
19	14	1	0	0
Abstain: 11				

Poll outcome:  consensus

Salient comments:

- **Favor:** It is amazing that the new comparison rules continue to cause so much trouble after so much time. The examples given show that these revised rewrite rules are at the very least a significant improvement and should reduce the pain of updating existing code to newer language standards.
- **Favor:** This is a necessary update to C++20. Migration to C++20 did sometimes cost a lot of time due to the various issues mentioned in the paper.
- **Strong Favor:** Given that implementations are doing something along those lines to avoid breaking existing code already - ship it.

§ 5.3. P2327r1 De-deprecating `volatile` compound operations

- [\[P2327r1\]](#)
- [GitHub issue](#)
- **Highlight:** C++20 deprecated many uses for `volatile`. This paper un-deprecates the bitwise compound operators (`|=`, `&=`, and `^=`) on `volatile` left operands.
-  **Poll:** Forward P2327r1 "De-deprecating `volatile` compound operations" to Core for C++23. Treat it as a Defect Report against C++20.

Poll votes:

SF	F	N	A	SA
13	19	1	0	2
Abstain: 10				

Poll outcome:  consensus

Salient comments:

- **Strong Favor:** The proposal retains compatibility with large swaths of often vendor-supplied hardware support libraries targeted at C. This makes C++ more viable in the embedded world. Programming patterns that embedded systems engineers are familiar with keep on working.
- **Strong Favor:** We want to increase the usage of C++ for embedded code; *not* doing this would alienate some users, and add more support for a "C++ is bad for embedded" position
- **Favor:** I can understand the problems caused by the C++20 change in that respect; the paper, however, does not fully address (it seems to me) the problems that led to the deprecation in the first place (the paper `_does_` explain how these operations can be well used under specific circumstances and by specific parties, of course, but use of these operations is not limited to such parties and circumstances). I'm split, but will err on the side of trusting users...
- **Favor:** It is a misfeature of the language that volatility pertains equally to reads and writes despite the fact that memory-mapped devices are often simplex. It is the correct balance to allow programmers to idiomatically use objects associated with such devices even though, even on hardware that supports an appropriately atomic modification, the language does not provide a mechanism for the compound assignment operators to support full-duplex communication.
- **Strong Against:** To me, the semantic of the compound assignment is not well defined in the standard. It says: `E1 op= E2` is equivalent to `E1 = E1 op E2` except that `E1` is evaluated only once. This is not enough to define volatile semantics. We need to know how many read access and how many write access occur, in what order. The standard was unclear before, so be it. It was cleaned, that was good. Reintroducing inaccurately defined construct is not the right way

to go. I would be in favor if the semantics were defined. The paper claims that deprecation (and eventually removal) will force developers to change the way they write embedded code. This is not true. Embedded compilers would probably support this construct as an extension, which is the right thing to do for something that has no clearly specified semantic (and I hope they will define the semantic they selected in their documentation).

- **Strong Against:** It is evident from discussion in and outside the committee that, while people are hurt by that depreciation, the semantics of compound assignments are too often assumed to be atomic or are otherwise unclear. The motivation for deprecating them hasn't disappear. It is important that the C committee and the industry follows suite with that - and it might take time. However, undeprecating them sends the wrong message, and still leaves users with the bugs the original paper was trying to correct.

§ 5.4. P2266r2 Simpler implicit move

- [\[P2266r2\]](#)
- [GitHub issue](#)
- **Highlight:** In C++20, `return` statements can *implicitly move* from local variables of *rvalue* reference type; but a defect in the wording means that *implicit move* fails to apply to functions that return references. C++20's implicit move is specified via a complicated process involving two overload resolutions, which is hard to implement, causing implementation divergence. We fix the defect and simplify the spec by saying that a returned move-eligible *id-expression* is always an *xvalue*.
- **Note:** r1 of this paper went through electronic polling, received mixed feedback based on new information, see [\[P1018r13\]](#) for the poll comments. It was updated to r2, re-discussed in the [2021-10-07 teleconference](#) to address the new information, and is now being re-pollled.
- 🗳️ **Poll:** Forward P2266r2 "Simpler implicit move" to Core for C++23.

Poll votes:

SF	F	N	A	SA
15	13	1	2	0
Abstain: 14				

Poll outcome:  consensus

Salient comments:

- **Favor:** Reading through the rationale presented in the paper, I find that these rules are consistent with thinking of r-value references as "owning" in the sense that they represent a promise that modifications to the referenced object will not be observed except by components to which ownership is subsequently transferred. These rules changes preserve that intuitive property without requiring extra syntax.
- **Favor:** It's unfortunate to break any existing code doing something as simple as returning a local variable, but even within that set many cases are plainly bugs or were already broken by C++20. Given that the language has a means of identifying Xpiring values in the type system, it seems only appropriate to use it in such a particularly straightforward context.
- **Favor:** The current "implicit move" semantics for a return statement are unlike any other overload resolution mechanism in the language. So this seems like a good direction. I would have preferred also treating this as a defect report.
- **Favor:** "It is a simpler rule, but probably will have more downstream consequences than the ones presented here. It's hard to know what those are. I think we should really invest, as an organization, in the ability to test changes like this. If we had an implementation of `==`, and the ability to test this on many large code bases with test suites, we probably would've never had the problem that P2468 now has to solve. Here, we do have an implementation of this paper, and we have some feedback of its impact. But to what extent is this a thorough-enough testing? It'd be nice if we could actually provide a good answer to that question in a way that could give us the confidence to say "Yes, this breaks some code, in these situations, and that is okay." Today, the best I can do is vote FOR and hope that no further, larger issues will pop up."
- **Against:** Weakly against P2266 (vs strongly before). There hasn't been any glut of new errors, and the problems seem to have "settled" from the Clang implementation. That said, the throw-example still gives me heartburn, otherwise I have no implementation concerns.

- **Against:** This broke code in STL headers written by C++ experts; that gives me no confidence it won't also break user code in the wild, which may be written by less-expert folks. Thankfully, the breakages we saw at Intel finally stopped, which is why this is an Against instead of a Strongly Against (but I'm borderline strong on this).

§ 5.5. P1467r8 Extended floating-point types and standard names

- [\[P1467r8\]](#)
- [GitHub issue](#)
- **Highlight:** In addition to the three standard floating-point types, `float`, `double`, and `long double`, implementations may define any number of *extended floating-point types*, similar to how implementations may define extended integer types. An extended floating-point type may have the same representation and the same set of values as a standard floating-point type. But the extended floating-point type is still a separate type, and is not just an alias for the standard type. The proposal covers conversion rank, promotion, implicit conversion, usual arithmetic conversion, narrowing conversion, overload resolution, literal suffixes. It also contains library changes.
-  **Poll:** Forward the language changes in P1467r8 "Extended floating-point types and standard names" to Core for C++23, assuming that Library Evolution will separately forward the library changes to Library for C++23.

Poll votes:

SF	F	N	A	SA
11	15	2	1	0
Abstain: 16				

Poll outcome:  consensus

Salient comments:

- **Favor:** While the feature as proposed still has some rough edges (e.g., imperfect C compatibility and complicated overload resolution behavior), the numerous reviews have plainly identified the least problematic means of integrating several new fundamental types into the language. Those types are of increasing practical importance and have the hardware support appropriate for a language feature.
- **Favor:** This will allow C++ applications to take advantage of existing and yet-to-come floating-point hardware in a standard way.
- **Against:** The lack of implementation experience of the proposed overload resolution rules is concerning.

§ 5.6. P2350r2 `constexpr class`

- [\[P2350r2\]](#)
- [GitHub issue](#)
- **Highlight:** Allow `constexpr` in the `class`-head, declaring that all member functions, including special member functions, in this `class` are implicitly `constexpr`.
-  **Poll:** Forward P2350r2 "`constexpr class`" to Core for C++23.

Poll votes:

SF	F	N	A	SA
12	12	9	5	2
Abstain: 5				

Poll outcome:  no consensus

Salient comments:

- **Strong Favor:** The standard library is moving towards "constexpr all the things" as much as it makes sense. We're at the point where slapping a "constexpr" at every declaration in a class becomes a nuisance and mental burden. This

should be avoided by moving the constexpr-ness of all direct class members up at the class level. Stuff that's not constexpr will then stand out as something to possibly reconsider in the design.

- **Strong Favor:** The code using this is way friendlier to read and maintain. The infrastructure was already introduced, so it is easy to teach and grok. If P2448 is accepted at the same time I see no downside.
- **Favor:** Useful to avoid a lot of boilerplate in modern code bases.
- **Against:** The value is IMO minimal, and I is going in the wrong direction of where constexpr is headed. Moving the annotation to the class makes the formerly "obvious" annotation less obvious.
- **Against:** Allowing `struct X constexpr ...` but not `struct X consteval ...` is suspect.
- **Against:** P2448 removes some of the important caveats to using this feature, but it remains a clever workaround for one of the defaults being wrong rather than a meaningful feature. The constexpr status of static data members seems orthogonal at best, such that this convenience encourages silly things like adding a base class to hold any non-constexpr ones.
- **Strong Against:** This goes into the wrong direction. Faced with the fantastic problem of everything being constexpr, WG21 should rather consider whether the keyword is useful at all. As pointed out by P2448, the constexpr keyword already offer no guarantee, and if it doesn't then maybe everything that can be called in a constant expression should be, and as such maybe an implementation should always try to evaluate member function calls at constexpr. The feature proposed by P2350 is a short term solution that only works in very narrow cases, and adding a non constexpr function forces to modify the whole class.
- **Strong Against:** Applying constexpr on static member functions turns this feature into a different one. Now users will start open classes with all static member functions to write constexpr libraries because there is no other way in the language to mark a series of free functions with constexpr.

§ 5.7. P1169r3 `static operator()`

- [\[P1169r3\]](#)
- [GitHub issue](#)
- **Highlight:** Allow declaring `operator()` as `static`, as well as making lambdas `static`.
- 🗳️ **Poll:** Forward P1169r3 "`static operator()`" to Core for C++23.

Poll votes:

SF	F	N	A	SA
16	16	3	1	1
Abstain: 8				

Poll outcome: ✅ consensus

Salient comments:

- **Strong Favor:** It is a little weird to exploit the ability to call static member functions via member-access syntax to allow a performance improvement, but the feature is valuable and the semantics are the only ones possible. Yet another default is now wrong (for captureless lambdas), although those are so often inlined that the practical impact may be small.
- **Strong Favor:** I have thought that this was possible many times only to find out (again) that it is not. Finally it will be.
- **Favor:** Sheds needless overhead.
- **Favor:** Seems generally useful. The restriction against static here seems arbitrary.
- **Against:** It's a patch
- **Strong Against:** This adds a lot of subtlety for very little gain in practice.

§ 5.8. P1774r5 Portable assumptions

- [\[P1774r5\]](#)
- [GitHub issue](#)

- **Highlight:** Add the `[[assume(expression)]]` attribute. The use of assumptions is intended to allow implementations to assume that a given expression is true, and to optimize the program based on that assumption.
- 🗳️ **Poll:** Forward P1774r5 "Portable assumptions" to Core for C++23.

Poll votes:

SF	F	N	A	SA
14	17	3	4	0
Abstain: 7				

Poll outcome:  consensus

Salient comments:

- **Strong Favor:** There are use cases where assumptions can significantly improve performance and decrease code size, and they have been existing practice on all major compilers for many years, but in a non-standardised, non-unified, non-portable, not-very-well-defined way. This paper fixes it. I am strongly in favour of including this in C++23.
- **Strong Favor:** Useful tool for code optimization, impressive usage examples for the standard library
- **Against:** The specification of what this actually DOES is too handwavy for my comfort. I'm still not sure what the limitations being placed on the implementation are, and the paper's assumption (hah!) of how the current implementations of similar things seem to be misguided.
- **Against:** This tool is too sharp and too seductive. I expect it to introduce many new UB in user code, any many cases without measurable benefits.
- **Against:** high potential for overuse leading to serious errors

§ 5.9. P1494r2 Partial program correctness

- [\[P1494r2\]](#)
- [GitHub issue](#)
- **Highlight:** Adds `std::observable()` (*name pending LEWG review*) which establishes an observable checkpoint, inhibiting "time travel" of Undefined Behavior past the checkpoint.
- 🗳️ **Poll:** Forward P1494r2 "Partial program correctness" to Core for C++23, pending LEWG approval.

Poll votes:

SF	F	N	A	SA
10	7	3	4	1
Abstain: 20				

Poll outcome:  no consensus

Salient comments:

- **Strong Favor:** We need this to properly reason about asserts and contracts. Until we get full contracts this will help us to get the correct behaviour in user build solutions
- **Favor:** This is potentially useful, though I think its usefulness in practical terms is quite limited, as it's hard to imagine why I would reach for this unless I already thought I had UB that I needed to protect against -- in which case I would rather fix the UB. I could see myself using this feature while actively iterating on code, but I doubt I'd ever commit code with this in it.
- **Favor:** This proposal seems pretty wild, but I'm curious to see where it goes.
- **Favor:** This seems like a useful - if expert-only - feature.
- **Favor:** Useful, although I'm sure it will get misunderstood and misused.
- **Neutral:** In the war between programmer implicit assumptions, and what the compiler can observe, I am not confident this helps more than other rewrites that acknowledge UB that the compiler has noted.

- **Against:** I waffle back and forth between liking the idea and having no confidence we have any idea how well this will work in practice because there is zero implementation experience.
- **Against:** This seems to lack real implementation experience.
- **Strong Against:** This solves a problem.... that users don't know exist, in a way users won't know about. This is a very expert-friendly solution, for a very narrow set of experts. And unlike other esoteric language feature, this paper doesn't provide specific use cases where such an expert could significantly improve a program using this feature. Are we doing anything more than *technically* providing a solution? Where in my code, should i put that? I can only assume every other line of code i write triggers some UB, and so, should i add `std::observable` calls everywhere? The paper seems to be greatly concerned about I/O, was stating that returning from I/O functions constitute checkpoints considered?

§ 5.10. P2448r0 Relaxing some `constexpr` restrictions

- [\[P2448r0\]](#)
- [GitHub issue](#)
- **Highlight:** Remove two rules about `constexpr` that make code ill-formed or ill-formed (no diagnostic required) when functions or function templates are marked `constexpr` that might never evaluate to a constant expression.
- 🗳️ **Poll:** Forward P2448r0 "Relaxing some `constexpr` restrictions" to Core for C++23. Treat it as a Defect Report.

Poll votes:

SF	F	N	A	SA
20	12	1	2	1
Abstain: 9				

Poll outcome:  consensus

Salient comments:

- **Strong Favor:** Seems to go against the spirit of `constexpr` rules of just diagnosing things that aren't actually wrong, with the added harm that the rules aren't consistently enforced.
- **Favor:** This improves the usability of `constexpr` and the language as a whole. Implementation is trivial.
- **Favor:** This change is consistent with the general principle of diagnosing `constexpr` violations as they occur, rather than trying to predict whether or not a code path conforms to `constexpr` requirements. This is much more useful for programmers, particularly those who need to support multiple language dialects.
- **Neutral:** I cannot make my mind if helping experts that write code that works seamlessly with several versions of the standard is more or less important than helping beginners that may have just badly written a `constexpr` function... I would be in favor of beginners (and vote against this) if I trusted diagnostics to work well in many non-toy examples, but since I don't, I'll be neutral.
- **Against:** This paper removes any amount of value to the `constexpr` keyword; which is that the compiler will make SOME effort to validate its usability as `constexpr`. I don't like at all that this causes us to no longer catch some obvious mistakes in the user's code.
- **Strong Against:** Footgun. I do not agree with the logic that a function must be used to be checked for legality. Telling the user "this can NEVER be correct" is a powerful feature in the real world, where not every user tests every function immediately upon writing it. Why make them write the test when the compiler can tell them at compile time, which is the whole value-add for `constexpr` in the first place?

§ 5.11. P2437r0 Support for `#warning`

- [\[P2437r0\]](#)
- [GitHub issue](#)
- **Highlight:** C23 is adding support for `#warning` as a complement to `#error`. Add it to C++23 as well.
- 🗳️ **Poll:** Forward P2437r0 "Support for `#warning`" to Core for C++23.

Poll votes:

SF	F	N	A	SA
26	11	2	0	0
Abstain: 6				

Poll outcome:  consensus

Salient comments:

- **Favor:** Simple, straight forward, useful, retains compatibility with C, standardizes existing practice. Ship it.

§ 5.12. P2324r1 Labels at the end of compound statements (C compatibility)

- [\[P2324r1\]](#)
- [GitHub issue](#)
- **Highlight:** Labels are allowed at the end of compound statements in C, but not C++. For example `int main() { invalid_cpp: }`. This paper rectifies this minor divergence.
-  **Poll:** Forward P2324r1 "Labels at the end of compound statements (C compatibility)" to Core for C++23.

Poll votes:

SF	F	N	A	SA
20	11	3	0	0
Abstain: 11				

Poll outcome:  consensus

Salient comments:

- **Favor:** Improves compatibility with C23, removes an IMHO unnecessary restriction.

§ 5.13. P2071r1 Named universal character escapes

- [\[P2071r1\]](#)
- [GitHub issue](#)
- **Highlight:** C++ programmers have been able to portably use characters outside of the basic source character set in character and string literals using *universal-character-names* since C++ was standardized. For example the following UTF-8 string literal: `u8"\u0100\u0300"`. This proposal enables the above literal to be written using Unicode assigned names instead of Unicode code point values. The following is equivalent to the preceding C++ example `u8"\N{LATIN CAPITAL LETTER A WITH MACRON}\N{COMBINING GRAVE ACCENT}"`.
-  **Poll:** Forward P2071r1 "Named universal character escapes" to Core for C++23.

Poll votes:

SF	F	N	A	SA
12	14	3	3	1
Abstain: 12				

Poll outcome:  consensus

Salient comments:

- **Strong Favor:** Avoid specifying magic values in code.
- **Favor:** One would hope that many `\u` escape sequences are accompanied by comments explaining the character being selected. It is plainly superior to have the compiler map from the human-readable concept to the numeric one, just as it does for machine code. The size cost in the implementation appears to be acceptable.
- **Favor:** Unicode compatibility is extremely important, and this feature makes it easier for users to incorporate Unicode into their applications. I find the concerns about increased binary size of the compiler to be extremely underwhelming.

- **Neutral:** While I can see the usefulness, I'm not sure Unicode character escapes is something that will be used often enough to make up for the need of a dictionary in the compiler. Neutral simply because it is not an overly large dictionary.
- **Against:** Concern about the compiler space overhead.
- **Strong Against:** Having to add a few hundred KB to C++ tools (which are often only a few MB) is way too high of a cost for the relatively minor convenience this feature offers.

§ 5.14. P2295r5 Support for UTF-8 as a portable source file encoding

- [\[P2295r5\]](#)
- [GitHub issue](#)
- **Highlight:** The set of source file character sets is currently implementation-defined. This paper mandates that C++ compilers must accept UTF-8 as an input format. How the source file encoding is detected, which other input formats are accepted, and what the storage media is (e.g. files), remain implementation-defined.
- 🗳️ **Poll:** Forward P2295r5 "Support for UTF-8 as a portable source file encoding" to Core for C++23.

Poll votes:

SF	F	N	A	SA
20	9	3	0	0
Abstain: 13				

Poll outcome:  consensus

Salient comments:

- **Strong Favor:** This is the most important change we can make for Unicode support in C++. It allows for string literals to seamlessly incorporate UTF-8 code sequences without worrying about how those sequences will be interpreted across varying compilers.

§ 5.15. P2290r2 Delimited escape sequences

- [\[P2290r2\]](#)
- [GitHub issue](#)
- **Highlight:** Add an additional, clearly delimited syntax for octal, hexadecimal and universal character name escape sequences. The new syntaxes `\u{}`, `\o{}`, `\x{}` are usable in places where `\u`, `\x`, `\nnn` currently are.
- 🗳️ **Poll:** Forward P2290r2 "Delimited escape sequences" to Core for C++23.

Poll votes:

SF	F	N	A	SA
9	15	3	1	1
Abstain: 16				

Poll outcome:  consensus

Salient comments:

- **Strong Favor:** The current alternative (write every character as a 3-digit octal number) is user-hostile. This is a welcome addition.
- **Neutral:** It's not clear that having yet another syntax is preferable to having our current less ergonomic forms, especially given that `\U00012345` is not much longer than `\u{12345}` and `\u1234` isn't greedy and is shorter than almost all `\u{...}` values. The greedy `\x` is a bit more trouble, but it's very common to have "binary" escapes in long sequences; code generators that might need to generate a mixture of escapes and ordinary characters can of course handle the awkwardness in just one place.

- **Against:** `\u{...}` saves characters and no longer asks users to learn both `\u` and `\U`, so people will use them. The other two are unlikely to be popular. `{0x2c, 0x10, 0}` is about the same length and more straightforward comparing to `"\x{2c}\x{10}"`. For the beneficial parts, this paper seems unnecessary if we further extend on top of P2071 `\N{U+10034}`.
- **Strong Against:** Unnecessary

§ 5.16. P2362r3 Remove non-encodable wide character literals and multicharacter wide character literals

- [\[P2362r3\]](#)
- [GitHub issue](#)
- **Highlight:** C++ currently permits writing a wide character literal with multiple characters or characters that cannot fit into a single `wchar_t` codeunit. This paper makes these literals ill-formed.
- 🗳️ **Poll:** Forward P2362r3 "Remove non-encodable wide character literals and multicharacter wide character literals" to Core for C++23.

Poll votes:

SF	F	N	A	SA
14	12	2	1	0
Abstain: 16				

Poll outcome:  consensus

Salient comments:

- h
- **Favor:** Multicharacter literals in general are not usefully standardized, and the wide-character variants have no practical application, not least because they suffer from the inadequate `wchar_t` on Windows. That the change is a simplification to one of the ugliest parts of the standard is a bonus.
 - **Favor:** It's hard to imagine that any use of these sequences is not a bug.
 - **Against:** I don't know what this paper **enables**, and the issue still looks like a QoI to me.

§ 5.17. P2348r2 Whitespaces Wording Revamp

- [\[P2348r2\]](#)
- [GitHub issue](#)
- **Highlight:** Clarify what constitutes a new-line and a whitespace, using Unicode terminology.
- 🗳️ **Poll:** Forward P2348r2 "Whitespaces Wording Revamp" to Core for C++23.

Poll votes:

SF	F	N	A	SA
10	17	2	0	0
Abstain: 16				

Poll outcome:  consensus

Salient comments:

- **Strong Favor:** This sort of modernization is welcome, if only so that we can stop spending time thinking about these issues informally. Removing silly things like IFNDR for comments is plainly an improvement. Consistency with other practical languages is more important than following Unicode's suggestions for plain text when there (unfortunately) is disagreement.

§ 6. Polling Process

For each poll, participants are asked to vote one of:

- Strongly in favor
- In favor
- Neutral
- Against
- Strongly against

Participants also have the option to abstain from voting on a particular poll. They are asked to comment on each poll. This comment is mandatory, as it helps the chair determine consensus.

§ 7. Teleconferences

Here are the minutes for the virtual discussions that were held since the Prague meeting in February 2020:

1. 2020-04-09 [Issue Processing](#)
2. 2020-04-15 [Issue Processing](#)
3. 2020-04-23 [Issue Processing](#)
4. 2020-04-29 [Issue Processing](#)
5. 2020-05-07 [Issue Processing](#)
6. 2020-05-13 [Issue Processing](#)
7. 2020-05-21 [A General Property Customization Mechanism](#)—[P1393R0] (P1393 tracking issue)
8. 2020-06-10 [Reviewing Deprecated Facilities of C++20 for C++23](#)—[P2139R1] (P2139 tracking issue)
9. 2020-06-18 [C++ Identifier Syntax using Unicode Standard Annex 31](#)—[P1949R4] (P1949 tracking issue)
10. 2020-06-24 [Extended floating-point types and standard names](#)—[P1467R4] (P1467 tracking issue)
11. 2020-07-02 [Allow Duplicate Attributes, Attributes on Lambda-Expressions](#)—[P2156R0] (P2156 tracking issue) and [P2173R0] (P2173 tracking issue)
12. 2020-07-08 [Pointer lifetime-end zap and provenance, too](#)—[P1726R3] (P1726 tracking issue)
13. 2020-07-16 [Guaranteed copy elision for return variables](#)—[P2025R1] (P2025 tracking issue)
14. 2020-07-30 [Reviewing Deprecated Facilities of C++20 for C++23, Removing Garbage Collection Support](#)—[P2139R2] (P2139 tracking issue) and [P2186R0] (P2186 tracking issue)
15. 2020-08-05 [Transactional Memory Lite Support in C++](#)—[P1875R0] (P1875 tracking issue)
16. 2020-08-19 [Freestanding Language: Optional `::operator new`, Mixed string literal concatenation](#)—[P2013R2] (P2013 tracking issue) and [P2201R0] (P2201 tracking issue)
17. 2020-08-27 [Exhaustiveness Checking for Pattern Matching](#)—[P1371R3] (P1371 tracking issue)
18. 2020-09-02 [#embed - a simple, scannable preprocessor-based resource acquisition method](#)—[P1967R2] (P1967 tracking issue)
19. 2020-09-10 [A pipeline-rewrite operator](#)—[P2011R1] (P2011 tracking issue)
20. 2020-09-16 [Pattern matching: inspect is always an expression](#)—[P1371R3] (P1371 tracking issue)
21. 2020-09-24 [C++ Identifier Syntax using Unicode Standard Annex 31, Member Templates for Local Classes](#)—[P1949R6] (P1949 tracking issue) and [P2044R0] (P2044 tracking issue)
22. 2020-09-30 [Narrowing contextual conversions to bool, Generalized pack declaration and usage](#)—[P1401R3] (P1401 tracking issue) and [P1858R2] (P1858 tracking issue)
23. 2020-10-08 [Compound Literals, `if constexpr`](#)—[P2174R0] (P2174 tracking issue) and [P1938R1] (P1938 tracking issue)
24. 2020-10-14 [Inline Namespaces: Fragility Bites, Trimming whitespaces before line splicing](#)—[P1701R1] (P1701 tracking issue) and [P2223R0] (P2223 tracking issue)

25. 2020-10-22 [Issues Processing](#)
26. 2020-10-28 [Issues Processing](#)
27. 2020-11-05 [Deducing `this`—P0847R5 \(P0847 tracking issue\)](#)
28. 2020-11-19 [goto in pattern matching](#)
29. 2020-12-03 [auto\(x\): decay-copy in the language—P0849R5 \(P0849 tracking issue\)](#)
30. 2021-01-14 [Polls](#)
31. 2021-01-20 [Final polls preparation](#)
32. 2021-01-28 [P2012 Fix the range-based for loop](#)
33. 2021-02-03 [P2280 Using unknown references in constant expressions](#)
34. 2021-02-11 [P1974 Non-transient constexpr allocation using `propconst`](#)
35. 2021-02-17 [P2242 Non-literal variables \(and labels and `gotos`\) in `constexpr` functions](#)
36. 2021-02-25 [P1371 pattern matching: implementation experience](#) pattern matching now has fairly capable prototype implementation and in this session it will be both demonstrated and described. Bruno, the core implementer, will also field any implementation related questions surrounding the proposal. See the [Godbolt demo](#).
37. 2021-03-02 [P2279R0 We need a language mechanism for customization points](#) joint EWG+LEWG session
38. 2021-03-11 [P2285 Are default function arguments in the immediate context?](#)
39. 2021-03-17 [P2266 Simpler implicit move](#)
40. 2021-03-25 [P2128 Multidimensional subscript operator](#)
41. 2021-03-31 [P2036 Changing scope for lambda trailing-return-type, and P2287 Designated-initializers for base classes](#)
42. 2021-04-08 [Pattern matching identifier patterns syntax: `let` vs `auto` vs `none`](#) Introducing bindings inside pattern matching: Unqualified identifier within pattern matching must have well defined meaning. Published version of the paper (P1371R4) proposed using case keyword to disambiguate names between identifier expressions and introducing named binding. This direction raised some concerns within committee and authors want to propose another way of resolving the ambiguity. After the presentation authors would like to have a poll to secure committee approval on the proposed direction.
43. 2021-04-22 [P2334 Add support for preprocessing directives `elifdef` and `elifndef`, and P2246 Character encoding of diagnostic text](#)
44. 2021-04-28 [Final discussion of the polls, and P2066 Suggested draft TS for C++ Extensions for Minimal Transactional Memory](#)
45. 2021-05-06 [P2314 Character sets and encodings, and quick review of r2 for P2280 Using unknown references in constant expressions](#)
46. 2021-05-12 [P2255 A type trait to detect reference binding to temporary](#)
47. 2021-05-20 [P2025 Guaranteed copy elision for named return objects](#)
48. 2021-05-26 [P2041 Deleting variable templates](#)
49. 2021-06-03 [P1967 `#embed` — a simple, scannable preprocessor-based resource acquisition method](#)
50. 2021-06-14 [Joint session with LEWG on P2138 Rules of Design <=> Specification engagement](#)
51. 2021-06-23 [P1701 Inline Namespaces: Fragility Bites](#)
52. 2021-07-01 [P2360R0 Extend init-statement to allow alias-declaration, and P2290 Delimited escape sequences, and P2316 Consistent character literal encoding](#)
53. 2021-07-07 [P2392R0 Pattern matching using `is` and `as`](#)
54. 2021-07-21 [P2350 constexpr class](#)
55. 2021-07-29 [P1169 `static operator\(\)`](#)
56. 2021-08-04 [P2347 Argument type deduction for non-trailing parameter packs](#)
57. 2021-08-12 [P2355 Postfix fold expressions](#)
58. 2021-08-26 [P2295 Support for UTF-8 as a portable source file encoding and P2362 Remove non-encodable wide character literals and multicharacter wide character literals](#)
59. 2021-09-01 [P2414R1 Pointer lifetime-end zap proposed solutions](#)

60. 2021-09-15 [P2327 De-deprecating volatile compound assignment](#)
61. 2021-09-23 [P2277 Packs outside of Templates](#)
62. 2021-09-29 [P2012 Fix the range-based for loop](#)
63. 2021-10-07 [P2280 Using unknown pointers and references in constant expressions](#) && [P2266 Simpler implicit move](#)
64. 2021-10-13 [P2448 Relaxing some `constexpr` restrictions](#) && [P2350 `constexpr` class](#)
65. 2021-10-21 [P1467 extended floating-point](#)
66. 2021-10-27 [P2437 support for `#warning`](#)
67. 2021-11-04 [P1494 Partial program correctness](#)
68. 2021-11-10 [P2348 Whitespaces Wording Revamp](#) && [P2071 Named universal character escapes](#)
69. 2021-12-02 [P2324 Labels at the end of compound statements](#) && [P114R5 Portable assumptions](#)
70. 2021-12-08 [P1705 Enumerating Core Undefined Behavior \(short paper, unlikely that there's content for EWG, let SG12 send to CWG\)](#) && [P2468R0 The Equality Operator You Are Looking For](#)
71. 2021-12-16 [P1467R7 Extended floating-point types and standard names](#)

§ 8. Remaining Open Issues

The following table lists all remaining open issues referred to EWG by Core or Library. Some of them are ready to be polled but are held back from the polling periods to limit the number of polls in this round.

From	#	Title	Notes
Lib	[LWG2432]	initializer_list assignability	<code>std::initializer_list::operator=</code> [support.initlist] is horribly broken and it needs deprecation: <pre>std::initializer_list<foo> a = {{1}, {2}, {3}}; a = {{4}, {5}, {6}}; // New sequence is already destroyed.</pre> Assignability of <code>initializer_list</code> isn't explicitly specified, but most implementations supply a default assignment operator. I'm not sure what [description] says, but it probably doesn't matter. Proposed resolution: Edit [support.initlist] p1, class template <code>initializer_list</code> synopsis, as indicated: <pre>namespace std { template<class E> class initializer_list { public: [...] constexpr initializer_list() noexcept; initializer_list(const initializer_list&) = default; initializer_list(initializer_list&&) = default;</pre>

		<pre>initializer_list& operator=(const initializer_list&) = delete; initializer_list& operator=(initializer_list&&) = delete; constexpr size_t size() const noexcept; [...] }; [...] }</pre> LWG telecon appears to want a language change to disallow assigning a braced-init-list to an <code>std::initializer_list</code> but still permit move assignment of <code>std::initializer_list</code> objects. That is, <pre>auto il1 = {1,2,3}; auto il2 = {4,5,6}; il1 = {7,8,9}; // currently well-formed but dangles immediately; should be ill-formed il1 = std::move(il2); // currently well-formed and should remain so</pre> Meeting: Proposed resolution: <pre>initializer_list(const initializer_list&) = default; initializer_list(initializer_list&&) = default; [[deprecated]] initializer_list& operator=(const initializer_list&) = default; [[deprecated]] initializer_list& operator=(initializer_list&&) = default;</pre> SF F N A SA 0 3 12 0 0 JF emailed LEWG, to see if they have an opinion, no feedback. Asked LEWG chairs to schedule for a telecon. LWG discussed priority.
--	--	---

Lib	[LWG2813]	std::function should not return dangling references	If a std::function has a reference as a return type, and that reference binds to a prvalue returned by the function that it wraps, then the reference is always dangling. Because any use of such a reference results in undefined behaviour, the std::function should not be allowed to be initialized with such a callable. Instead, the program should be ill-formed. A minimal example of well-formed code under the current standard that exhibits this issue: <pre>int main() { std::function<const int&()> F([]{ return 42; }); int x = F(); // oops! }</pre> Proposed resolution: Add a second paragraph to the remarks section of 20.14.16.2.1 [func.wrap.func.con]: <pre>template<class F> function(F f);</pre> -7- Requires: F shall be CopyConstructible. -8- Remarks: This constructor template shall not participate in overload resolution unless • F is Lvalue-Callable (20.14.16.2 [func.wrap.func]) for argument types ArgTypes... and return type U, and • If R is type "reference to T" and INVOKE(ArgTypes..., f) has value category V and type U: ◦ V is a prvalue, U is a class type, and T is not reference-related (9.4.3 [dcl.init.ref]) to U, and ◦ V is an lvalue or xvalue, and either U is a class type or T is reference-related to U. [...] Tim: LWG in Batavia 2018 would like a way to detect when the initialization of a reference would bind to a temporary. This requires compiler support, since there's no known way in the current language to do this reliably in the presence of user-defined conversions (see thread starting at https://lists.isocpp.org/lib/2017/07/3256.php). Meeting: Tim wrote p2255 to address this. Ville thinks there should be an analysis of alternative approaches. Also see P0932.
-----	---------------------------	---	--

Core	[CWG2261]	Explicit instantiation of in-class friend definition	<pre>struct S { template <class T> friend void f(T) { } }; template void f(int); // Well-formed?</pre> A friend is not found by ordinary name lookup until it is explicitly declared in the containing name but declaration matching does not use ordinary name lookup. There is implementation divergence on handling of this example. Note 2020-04-29 Tentative agreement: This should be well-formed. SF 1 F 10 N 2 A 1 SA 0 JF emailed EWG / Core about this. Davis: the current name lookup approach which Core is taking in p1787 would disallow this. Support is possible, it would be inconsistent, but would also be a feature. Notes 2020-10-22: wait until p1787 is voted into the working draft, because it's making this behavior intentional. At that point, we can vote on marking the issue as Not a Defect. No objection to unanimous consent. Notes 2021-09-23: mark CWG2261 as Not A Defect, the example is now clearly ill-formed thanks to if we want to change this then we'd need a paper. <table><tr><td> </td><td>SF</td><td> </td><td>F</td><td> </td><td>N</td><td> </td><td>A</td><td> </td><td>SA</td><td> </td></tr><tr><td> </td><td>3</td><td> </td><td>8</td><td> </td><td>3</td><td> </td><td>0</td><td> </td><td>0</td><td> </td></tr></table>		SF		F		N		A		SA			3		8		3		0		0	
	SF		F		N		A		SA																
	3		8		3		0		0																
Core	[CWG2270]	Non-inline functions and explicit instantiation declarations	Hubert: question over the role of the inline keyword in relation to explicit instantiation declaration For inline functions, explicit instantiation declarations do not have the effect of suppressing implicit instantiation. A user's desire for wanting to suppress implicit instantiation can arise for different reasons: To reduce space in object files, executables, etc. and similarly to reduce the number of input symbols linker To reduce compile time in performing semantic analysis for instantiations whose definitions are provided elsewhere To control point-of-instantiation, avoiding contexts where the requisite declarations are not declared The special rule around inline functions allows inline-ness to be used to indicate that the first redeclaration is the intent and that instantiation-for-inlining is okay. Consider the following as a translation unit: <pre>template <typename T> //inline void f(T t) { g(t); } enum E : int; extern template void f(E); void h(E e) { f(e); }</pre> Marking the template definition inline would mean that the intended declaration for g would need to be provided as the best candidate at the points-of-instantiation for f<E>. The issue initially points out that this use of the inline keyword does not match a view that inline is essentially an ODR tool to allow multiple definitions (as opposed to a way to indicate desire for inlining).																						

			proposed that extern template merely has the effect of suppressing definitions in terms of linkage (regardless of the inline keyword). Such a change would affect the usability of the feature for user that falls within the latter two options above. I am not sure if CWG is asking EWG a specific question other than the general "we do not believe this wording or obvious consistency issue; is this an issue in terms of design?" Meeting: Hubert forked the thread on the reflector. Might want the education SG to take a look, or might want Meeting 2020-10-28: Inbal will try to put together the wording, to prove / disprove whether this is a c Notes 2021-09-23: POLL: in a new world with modules, inline isn't merely an ODR tool. We therefore believe that CWG2270 is Not a Defect. SF F N A SA 0 10 2 0 0
Core	[CWG794]	Base-derived conversion in member type of pointer-to-member conversion	Related to CWG 170, drafting by Clark seems unlikely. This is section 2.1 of Jeff Snyder's P0149R0, was approved by EWG, 4 years ago, waiting for wording. JF reached out to Jeff. Did wording with Jens, main blocker is lack of implementation. Meeting: (notes) Suggest closing as Not A Defect because we have implementation uncertainties, but explore the design space in P0149. ABI group will discuss. All in favor.
Core	[CWG900]	Lifetime of temporaries in range-based for	<pre>// some function std::vector<int> foo(); // correct usage auto v = foo(); for(auto i : reverse(v)) { std::cout << i << std::endl; } // problematic usage for(auto i : reverse(foo())) { std::cout << i << std::endl; }</pre> Meeting: (notes, also discussed in Rapperswil 2014) Suggest closing as Not A Defect because it's a c which might have effects on existing code (might cause bugs), and might need to change more than just range-based loops. See p0614, p0577, p0936, p1179. We'll explore the design space in a separate paper circled back with Nico on this, might write a paper. All in favor.
Core	[CWG1008]	Querying the alignment of an object	https://godbolt.org/z/_SN8iF • GCC already implements this extension without issuing a warning • Clang and EDG implement this extension for gcc compatibility, with a warning • MSVC does not yet implement this feature Quick example using 'auto' illustrates why we might want this capability for objects as well as types. Principle of least astonishment suggests it is surprising for sizeof and alignof to behave differently in regard. Recommend shipping this straight to core as soon as we can find a wording champion. We need to discuss with WG14. Questions for EWG to answer before forwarding:

			• Should this be a unary operator, like sizeof, so "alignof x" is valid? Or should it be like typeid, where parens are required? • What would this mean? Is it the alignment of the type of the object? Or the compiler's best guess: alignment of the expression itself? Should it take into account any facts that are known about the expression other than its type? If so, which ones? (For example, if applied to an expression x or x is declared with an alignas attribute, is that value returned?) This needs a design paper rather than going straight to core. https://godbolt.org/z/TeVA9T GCC seems to report the alignment of the object not just of decltype(object). Meeting: (notes, also discussed in Rapperswil 2014) Suggest closing as Not A Defect, the design is complex especially around alignment of object versus type. Invite a paper, Inbal will pitch in, Alidair can collate. All in favor. Inbal's paper: P2152R0
Core	[CWG1077]	Explicit specializations in non-containing namespaces	The current wording of 9.8.1.2 [namespace.memdef] and 13.9.3 [temp.expl.spec] requires that an explicit specialization be declared either in the same namespace as the template or in an enclosing namespace. It would be convenient to relax that requirement and allow the specialization to be declared in a non-enclosing namespace to which one or more of the template arguments belongs. Additional note, April, 2015: See EWG issue 48. Might allow us to revert DR2061 and all the horribleness that created and described in p1701. The paper p1077 addresses is the motivating factor in dr2061. Also see CWG 2370. Meeting: (notes) Suggest closing as Not A Defect. See p0665, minutes. Continue under p0665 or a future version of it. All in favor.
Core	[CWG1433]	trailing-return-type and point of declaration	<pre>template <class T> T list(T x); template <class H, class ...T> auto list(H h, T ...args) -> decltype(list(args...)); // list isn't in scope in its own trailing-return-type auto list3 = list(1, 2, 3);</pre> Meeting: (notes, also discussed in Rapperswil 2014) there might be compiler divergence according to Daveed. Suggest closing as Not A Defect, it would be tricky to change behavior without ambiguity. "If this would break existing code that relies on seeing only previous declarations. No objection to unanimous consent.
Core	[CWG1469]	Omitted bound in array new-expression	The syntax for nopttr-new-declarator in 7.6.2.7 [expr.new] paragraph 1 requires an expression, even though the bound could be inferred from a braced-init-list initializer. It is not clear whether 9.4.1 [dcl.init.aggr] paragraph 4, An array of unknown size initialized with a brace-enclosed initializer-list containing n initializers, where n shall be greater than zero, is defined as having n elements (9.3.3.4 [dcl.array]). should be considered to apply to the new-type-id variant, e.g., <pre>new (int[]){1, 2, 3}</pre> This was addressed by p1009 Meeting: (notes) Suggest closing as Not A Defect. Addressed by P1009. No objection to unanimous consent.

Core	[CWG1555]	Language linkage and function type compatibility	Currently function types with different language linkage are not compatible, and 7.6.1.2 [expr.call] paragraph 1 makes it undefined behavior to call a function via a type with a different language linkage. These features are generally not enforced by most current implementations (although some do) between functions with C and C++ language linkage. Should these restrictions be relaxed, perhaps as conditionally-supported features? Somewhat related to CWG1463. Meeting: (notes) no strong consensus at the moment, Erich Keane brought this up with SG12 Undefined Behavior. Long discussion, will need to revisit, need a paper. Meeting Oct 22nd 2020: will need a volunteer to write a paper, but the wording in the standard is unambiguous, and we have existence proof of platforms which use different calling conventions between C and C++. However many major compilers have chosen to ignore this. Not a defect, but we welcome a paper to change the status quo. No objection to unanimous consent.
Core	[CWG1643]	Default arguments for template parameter packs	Although 13.2 [temp.param] paragraph 9 forbids default arguments for template parameter packs, allowing them would make some program patterns easier to write. Should this restriction be removed? Meeting: (notes) Suggest closing as Not A Defect. Interesting design space, but needs a paper, see N1700. No objection to unanimous consent.
Core	[CWG1864]	List-initialization of array objects	The resolution of issue 1467 now allows for initialization of aggregate classes from an object of the same type. Similar treatment should be afforded to array aggregates. See recent discussion on allowing 'auto x[] = {1, 2, 3};' -- this topic came up there. The two questions were answered independently, but some consider them to be related. Meeting: (notes) Suggest closing as Not A Defect. Fixing anything in this space would require a paper that considers the entire design space, Timur might be interested in this. No objection to unanimous consent.
Core	[CWG1871]	Non-identifier characters in <i>ud-suffix</i>	JF forwarded to SG16 Unicode given their work on TR31. Tracked on GitHub. SG16 reviewed D194 and still needs wording changes. Discussed at the April 22nd SG16 telecon. SG16 Poll: Is there any objection to unanimous consent for recommending rejection of this proposal? No objection to unanimous consent. EWG Meeting: (notes 2020-04-15, notes 2020-05-07) Suggest closing as Not A Defect. No objection to unanimous consent.
Core	[CWG1876]	Preventing explicit specialization	A desire has been expressed for a mechanism to prevent explicitly specializing a given class template particular <code>std::initializer_list</code> and perhaps some others in the standard library. It is not clear whether simply adding a prohibition to the description of the templates in the library clauses would be sufficient or whether a core language mechanism is required. Meeting: (notes) Suggest closing as Not A Defect. No objection to unanimous consent. This could be a language feature, would need library usecase examples. Poll: we are interested in such a paper SF 2 F 13 N 6 A 1 SA 0
Core	[CWG1915]	Potentially-invoked destructors in non-throwing constructors	Since the base class constructor is non-throwing, the deleted base class destructor need not be referenced. There's a typo in the issues list here: it should say "Since the <i>derived</i> class constructor is non-throwing, the deleted base class destructor need not be referenced." Meeting: (notes 2020-04-23) the proposed wording changes the implementation leeway in two-phase unwinding, which breaks some existing ABIs. We would need a paper to explore this further. Suggest closing as Not A Defect. No objection to unanimous consent.

Core	[CWG1923]	Lvalues of type void	There does not seem to be any significant technical obstacle to allowing a void* pointer to be dereferenced and that would avoid having to use weighty circumlocutions when casting to a reference to an object designated by such a pointer. Might consider this a duplicate of the discussion on "regular void". JF reached out to Matt. He has an implementation in clang. Needs to update the paper. Might need a v to present. Daveed would be interested in presenting. Meeting: (notes 2020-04-23) Suggest closing as Not A Defect. Explore under the "regular void" umbrella. No objection to unanimous consent.
Core	[CWG1934]	Relaxing exception-specification compatibility requirements	According to 14.5 [except.spec] paragraph 4, If any declaration of a function has an exception-specification that is not a noexcept-specification all exceptions, all declarations, including the definition and any explicit specialization, of that function shall have a compatible exception-specification. This seems excessive for explicit specializations, considering that paragraph 6 applies a looser requirement for virtual functions: If a virtual function has an exception-specification, all declarations, including the definition, of any function that overrides that virtual function in any derived class shall only allow exceptions that are allowed by the exception-specification of the base class virtual function. The rule in paragraph 3 is also problematic in regard to explicit specializations of destructors and default special member functions, as the implicit exception-specification of the template member function cannot be determined. There is also a related problem with defaulted special member functions and exception-specifications. According to 9.5.2 [dcl.fct.def.default] paragraph 3, If a function that is explicitly defaulted has an explicit exception-specification that is not compatible ([except.spec]) with the exception-specification on the implicit declaration, then • if the function is explicitly defaulted on its first declaration, it is defined as deleted; • otherwise, the program is ill-formed. This rule precludes defaulting a virtual base class destructor or copy/move functions if the derived class member function will throw an exception not allowed by the implicit base class member function. Meeting: (notes 2020-04-23) JF will work with Mike to update wording to C++20. Will revisit. From Mike: This issue is NAD since we eliminated typed exception-specifications. The current wording dealing only with noexcept(true) and noexcept(false), does not have this issue. Will remove "extension" status. Meeting: no objection to CWG taking it back, and marking it as NAD.

Core	[CWG1957]	decltype(auto) with direct-list-initialization	Paper N3922 changed the rules for deduction from a braced-init-list containing a single expression in initialization context. Should a corresponding change be made for decltype(auto)? E.g., <pre>auto x8a = { 1 }; // decltype(x8a) is std::initializer_list<int></pre> decltype(auto) x8d = { 1 }; // ill-formed, a braced-init-list is not an expression <pre>auto x9a{ 1 }; // decltype(x9a) is int</pre> decltype(auto) x9d{ 1 }; // decltype(x9d) is int See also issue 1467, which also effectively ignores braces around a single expression, this change was parallel to that one, even though the primary motivation for decltype(auto) is in the return type of a forwarding function, where direct-initialization does not apply. Meeting: (notes 2020-04-23) Suggest closing as Not A Defect. This would be a language change, it's that we want to make such a change. It would require a paper. Mike Spertus is writing a paper with some overlap, will cover this as well. No objection to unanimous consent.
Core	[CWG2111]	Array temporaries in reference binding	The current wording of the Standard appears to permit code like <pre>void f(const char (&)[10]); void g() { f("123"); f({'a', 'b', 'c', '\0'}); }</pre> creating a temporary array of ten elements and binding the parameter reference to it. This is controversial and should be reconsidered. (See issues 1058 and 1232.) Meeting: (notes 2020-04-23) JF digging more, talking to Richard about this. Somewhat related to P2174 compound literals. Would be very strange if <pre>const char (&&x)[10] = {'a', 'b', 'c'};</pre> ... were invalid but ... <pre>const char x[10] = {'a', 'b', 'c'};</pre> <pre>const char (&&x)[3] = {'a', 'b', 'c'};</pre> ... were both OK. Maybe either we should allow the trailing elements to be zeroed in general (the status quo) or not (a major breaking change). Which means we should reject the issue on consistency grounds. The general rule is that if <pre>T &&r = init;</pre> ... can't bind directly, we create a temporary initialized as if with <pre>T r = init;</pre> ... and bind to that. (And similarly for const references.) Another question: Do we want to support (T){inits} as a synonym for T{inits}? Meeting Oct 22nd 2020: Note that the issue itself is defective, and <code>f("123")</code> isn't valid.

			CWG2352 references p1358, and was resolved, leading us to believe that this is now mainstream and controversial. Not a defect. No objection to unanimous consent.
Core	[CWG2125]	Copy elision and comma operator	Currently, <code>_N4750_.15.8</code> [class.copy] paragraphs 31-32 apply only to the name of a local variable in determining whether a return expression is a candidate for copy elision or move construction. Would sense to extend that to include the right operand of a comma operator? <pre> X f() { X x; return (0, x); } </pre> Meeting: (notes 2020-04-23) Consider expanding to other places that expand bit-field-ness such as <code>throw : x;</code>. Suggest closing as Not A Defect. Will need a paper to address, no current volunteer for this objection to unanimous consent.
Core	[CWG2132]	Deprecated default generated copy constructors	EWG has indicated that they are not currently in favor of removing the implicitly declared defaulted constructors and assignment operators that are deprecated in <code>_N4750_.15.8</code> [class.copy] paragraphs 7 & 8. Should this deprecation be removed? Related: discussing under p2139 deprecations. Meeting: (note 2020-04-29) Suggest closing as Not A Defect. No objection to unanimous consent. We want to remove entirely, or un-deprecate. This will need a paper, Ville will talk with Alisdair about for the topic from p2139.
Core	[CWG914]	Value-initialization of array types	Although value-initialization is defined for array types and the <code>()</code> initializer is permitted in a mem-init naming an array member of a class, the syntax <code>T()</code> (where <code>T</code> is an array type) is explicitly forbidden by [expr.type.conv] paragraph 2. This is inconsistent and the syntax should be permitted. Rationale (July, 2009): The CWG was not convinced of the utility of this extension, especially in light of questions about handling the lifetime of temporary arrays. This suggestion needs a proposal and analysis from the EWG before it can be considered by the CWG. This has become a more severe inconsistency after we adopted Ville's P0960 for C++20. Now it's not just <code>T()</code> that has this weird special-case restriction, it's <code>(a, b, c)</code> too: <pre> using X = int[]; X x{1, 2, 3}; // ok, int[3] X y(1, 2, 3); // ok, int[3] f(X{1, 2, 3}); // ok, int[3] temporary f(X(1, 2, 3)); // error </pre> Meeting: (notes) it is a defect, need a paper. David Stone trying to find a volunteer to write said paper favor.
Core	[CWG1463]	extern "C" alias templates	Currently 13 [temp] paragraph 4 forbids any template from having C linkage. Should alias templates be exempt from this prohibition, since they do not have any linkage? Additional note, April, 2013: It was suggested (see messages 23364 through 23367) that relaxing this restriction for alias templates could provide a way of addressing the long-standing lack of a way of specifying C linkage for non-alias templates.

			a language linkage for a dependent function type (see issue 13). The priority was deleted to allow CW consider the implications of that potential application of the facility. We should either have some way to express a dependent function type with C language linkage (and s accept 1555 below) or we should remove the notion that C language linkage (or not) is part of the fun type at all. (Apparently some targets used it; are they still in use? EDG probably knows.) Actively discussed on CWG reflector. Davis' interpretation of CWG discussion: I don't speak for Core, of course, but my (Evolutionary) thoughts are those given (last) in the messag which you replied: neither the wording nor the apparent intent of [temp.pre]/6's restrictions on C link templates make any sense. Since templates (and explicit specializations) can't have C linkage of the mangling variety (which the standard calls language linkage of a (function) name) but can have C lin the calling-convention variety (which the standard calls language linkage of a (function) type), extern should grant them the latter and not the former with no error. This has certain obvious applications in C APIs with callbacks. (Put differently, CWG13 shouldn't have been rejected; it appears to have gott bogged down in questions of syntax, but we have an adequate syntactic workaround that we merely h permit.) CWG1463 asks for a (very) proper subset of the above: merely that extern "C" be allowed to apply to templates (claiming incorrectly that they have no name with linkage at all). I consider it more releva more productive) to talk about whether entities have names with language linkage than with the ordin kind. The Core reflector discussion (which seems to have finished for the moment) also touched on the cas "the same" function template is declared with two different language linkages for its type, but the onl relevant Evolution input there would be a decision to go the opposite way and forbid function templai having types with C language linkage entirely. (They currently can, but it's mostly or entirely useless If (for CWG1555, also on your list) the rules about language linkage of (function) types are sufficien relaxed, then CWG1463 may be moot (and my extension of it with it), but that seems unlikely given l recent identification of a case where they matter. Meeting: (notes 2020-04-15, notes 2020-10-22, also discussed in Rapperswil 2014) We agree that thi issue. extern "C" on a template should only affect calling convention, and not the mangling. Davis an Hubert volunteer to write a paper. Unanimous consent.
Core	[CWG1790]	Ellipsis following function parameter pack	Although the current wording permits an ellipsis to immediately follow a function parameter pack, it clear that the <ctdarg> facilities permit access to the ellipsis arguments. The problem here (which is not explained in the issue) is: how do you supply the name of the last par before the ellipsis to va_start? You can't put the name of a pack there (it wouldn't be expanded) and t no way to name the last element of the pack (nor to deal with the case where the pack is empty). Meeting: (notes) 3 options: fix wording around "last parameter", remove facility entirely (either va_s function declarator), try to invent a language facility. JF emailed EWG to see if anyone has a strong preference, or if we should send back to CWG to fix wording, long discussion. Michael Spertus: I am willing to commit to including and analysis on this in an upcoming paper on parameter packs. JF followed up with Michael and Barry, no response.
Core	[CWG1962]	Type of __func__	Two questions have arisen regarding the treatment of the type of the __func__ built-in variable. First, implementations accept <pre>void f() {</pre>

			<pre>typedef decltype(__func__) T; T x = __func__; }</pre> even though T is specified to be an array type. In a related question, it was noted that <code>__func__</code> is implicitly required to be unique in each function, and not only the value but the type of <code>__func__</code> are implementation-defined; e.g., in something like <pre>inline auto f() { return &__func__; }</pre> the function type is implementation-specific. These concerns could be addressed by making the value prvalue of type <code>const char*</code> instead of an array lvalue. Rationale (November, 2018): See also issue 2362, which asks for the ability to use <code>__func__</code> in a <code>constexpr</code> function. These two goals are incompatible. The deep question here is about <code>__func__</code> and the ODR. Does EWG want implementations to somehow behave as if <code>__func__</code> is the same in all copies of an inline function (in which case it can have an address and be usable in constant expressions, but the <i>demangling</i> algorithm used to construct it becomes part of the ABI), or does EWG want implementations to behave as if <code>__func__</code> may differ between copies, so its address is not known until runtime (in which case it must have either pointer or incomplete array type, and its value is not usable in constant expressions -- but its address could still be usable). Note: C++20 has <code>std::source_location::function_name</code>. Meeting: (notes 2020-04-23) We should discuss this with WG14. This is indeed a language issue. No objection to unanimous consent. We'll need a paper, potentially considering what Reflection could do brought it up on the mailing list. We probably need a paper to disentangle this.
Core	[CWG2228]	Ambiguity resolution for cast to function type	Proposed resolution. C++ has a blanket disambiguation rule that says if a sequence of tokens can be interpreted as either a name or an expression, the type-name interpretation is chosen. This is unhelpful in cases like <pre>(T())+3</pre> where the current rules make <code>"(T())"</code> a cast to the type "function with no parameters returning T" rather than a parenthesized value-initialization of a temporary of type T. The former interpretation is always ill-formed, while the latter could be useful. Richard's proposed resolution, cited above, is to avoid the ambiguity by changing the grammar so that <code>"(T())"</code> cannot be parsed as a cast. The wording in the proposal applies that change to a number of different contexts where the ambiguity can come into play. There are two contexts where the change is applied, however: 1) the operand of <code>typeid</code>, and 2) a template-argument. During our discussion yesterday there was some support for the idea of applying the change to <code>typeid</code>, as well, although that would be a breaking change for any programs that rely on the current disambiguation to get type information for function types. There was general agreement, however, to exclude template arguments, since they are for things like <code>std::function</code>.

			CWG would, therefore, like some guidance from EWG on two questions. First, should we apply the r syntax to the operand of typeid, even though it's a breaking change? More generally, the question was raised whether we should make this change at all. Although resolving ambiguity in the other direction would arguably be more convenient in many cases, there is a tension that convenience and the complexity of the language. In particular, we would be creating a situation where the exact same sequence of tokens would mean two different things, depending on the context in which they appear. Is the convenience worth the cost in complexity? Meeting: (note 2020-04-29, notes 2020-03-23, note from Core summer 2020, notes from Core Prague 2019, notes from Core 2019-01-07, notes from Core Kona 2019, notes from Core Cologne 2019) This is an issue we'd like to change the standard to resolve it: SF 0 F 6 N 4 A 3 SA 2 Davis emailed EWG reflector. Long discussion.
Core	[CWG2296]	Are default argument instantiation failures in the "immediate context"?	Example 1: <pre>template <typename T, typename U = T> void fun(T v, U u = U()) {} void fun(...) {} struct X { X(int) {} // no default ctor }; int main() { fun (X(1)); }</pre> Consider the following example (taken from issue 3 of paper P0348R0): Example 2: <pre>template <typename U> void fun(U u = U()); struct X { X(int) {} }; template <class T> decltype(fun<T>()) g(int) { } template <class> void g(long) { } int main() { g<X>(0); }</pre> When is the substitution into the return type done? The current specification makes this example ill-formed because the failure to instantiate the default argument in the decltype operand is not in the immediate context of the substitution, although a plausible argument for making this a SFINAE case can be made. Meeting: (notes 2020-05-07) The first example under issue 3 of paper P0348R0 should become well-formed. SF F N A SA 0 1 4 3 6

			The second example under issue 3 of paper P0348R0 should become well-formed. SF F N A SA 1 5 7 3 0 This is an issue. We'd like to see a paper addressing it. It should explore what "Immediate context" m No objection to unanimous consent. Daveed / Hubert might entertain writing a paper explaining this. Ville emailed EWG about this. JF contacted Andrzej to see if he's interested in addressing this. Not sure he is. Meeting 2020-10-28: Tomasz will write a paper which exposes the issue and what he thinks should b (but not resolving wording itself). Hubert remembers an email about this, will find it and sync with JF Meeting 2021-01-14: Andrzej wrote a paper for this (emailed 2021-01-12 to EWG).
Core	[CWG2362]	<code>__func__</code> should be constexpr	The definition of <code>__func__</code> in 9.5.1 [dcl.fct.def.general] paragraph 8 is: <pre>static const char __func__[] = "function-name";</pre> This prohibits its use in constant expressions, e.g., <pre>int main () { // error: the value of __func__ is not usable in a constant expression constexpr char c = __func__[0]; }</pre> Rationale (November, 2018): See also issue 1962, which asks that the type of <code>__func__</code> be <code>const char</code> two goals are incompatible. Meeting: handle with 1962.

§ 9. Near-future EWG plans

We will continue to work on issue resolution, handle any C++23 fixes, start working on C++26, prioritizing according to [\[P0592R4\]](#).

§ References

§ Informative References

[CWG1008]

Steve Clamage. [Querying the alignment of an object](#). 27 November 2009. extension. URL: <https://wg21.link/cwg1008>

[CWG1077]

Mike Spertus. [Explicit specializations in non-containing namespaces](#). 13 June 2010. extension. URL: <https://wg21.link/cwg1077>

[CWG1433]

Jason Merrill. [trailing-return-type and point of declaration](#). 20 December 2011. extension. URL: <https://wg21.link/cwg1433>

- [CWG1463]
Daveed Vandevorde. [extern "C" alias templates](https://wg21.link/cwg1463). 19 August 2011. extension. URL: <https://wg21.link/cwg1463>
- [CWG1469]
Johannes Schaub. [Omitted bound in array new-expression](https://wg21.link/cwg1469). 12 February 2012. extension. URL: <https://wg21.link/cwg1469>
- [CWG1555]
Daniel Krüger. [Language linkage and function type compatibility](https://wg21.link/cwg1555). 18 September 2012. extension. URL: <https://wg21.link/cwg1555>
- [CWG1643]
Vinny Romano. [Default arguments for template parameter packs](https://wg21.link/cwg1643). 17 March 2013. extension. URL: <https://wg21.link/cwg1643>
- [CWG1790]
Daryle Walker. [Ellipsis following function parameter pack](https://wg21.link/cwg1790). 1 October 2013. extension. URL: <https://wg21.link/cwg1790>
- [CWG1864]
Alisdair Meredith. [List-initialization of array objects](https://wg21.link/cwg1864). 15 February 2014. extension. URL: <https://wg21.link/cwg1864>
- [CWG1871]
Richard Smith. [Non-identifier characters in ud-suffix](https://wg21.link/cwg1871). 17 February 2014. extension. URL: <https://wg21.link/cwg1871>
- [CWG1876]
John Spicer. [Preventing explicit specialization](https://wg21.link/cwg1876). 19 February 2014. extension. URL: <https://wg21.link/cwg1876>
- [CWG1915]
Aaron Ballman. [Potentially-invoked destructors in non-throwing constructors](https://wg21.link/cwg1915). 15 April 2014. extension. URL: <https://wg21.link/cwg1915>
- [CWG1923]
Richard Smith. [Lvalues of type void](https://wg21.link/cwg1923). 6 May 2014. extension. URL: <https://wg21.link/cwg1923>
- [CWG1934]
Vinny Romano. [Relaxing exception-specification compatibility requirements](https://wg21.link/cwg1934). 3 June 2014. extension. URL: <https://wg21.link/cwg1934>
- [CWG1957]
Vinny Romano. [decltype\(auto\) with direct-list-initialization](https://wg21.link/cwg1957). 20140630. extension. URL: <https://wg21.link/cwg1957>
- [CWG1962]
Steve Clamage. [Type of __func__](https://wg21.link/cwg1962). 4 July 2014. extension. URL: <https://wg21.link/cwg1962>
- [CWG2111]
Vinny Romano. [Array temporaries in reference binding](https://wg21.link/cwg2111). 2 April 2015. extension. URL: <https://wg21.link/cwg2111>
- [CWG2125]
Vinny Romano. [Copy elision and comma operator](https://wg21.link/cwg2125). 6 May 2015. extension. URL: <https://wg21.link/cwg2125>
- [CWG2132]
Nico Josuttis. [Deprecated default generated copy constructors](https://wg21.link/cwg2132). 28 May 2015. extension. URL: <https://wg21.link/cwg2132>
- [CWG2228]
Richard Smith. [Ambiguity resolution for cast to function type](https://wg21.link/cwg2228). 2 February 2016. review. URL: <https://wg21.link/cwg2228>
- [CWG2261]
Jens Maurer. [Explicit instantiation of in-class friend definition](https://wg21.link/cwg2261). 28 April 2016. extension. URL: <https://wg21.link/cwg2261>
- [CWG2270]
Richard Smith. [Non-inline functions and explicit instantiation declarations](https://wg21.link/cwg2270). 10 June 2016. extension. URL: <https://wg21.link/cwg2270>
- [CWG2296]
Jason Merrill. [Are default argument instantiation failures in the “immediate context”?](https://wg21.link/cwg2296). 25 June 2016. extension. URL: <https://wg21.link/cwg2296>
- [CWG2362]
Anthony Polukhin. [__func__ should be constexpr](https://wg21.link/cwg2362). 23 October 2017. extension. URL: <https://wg21.link/cwg2362>
- [CWG794]
CH. [Base-derived conversion in member type of pointer-to-member conversion](https://wg21.link/cwg794). 3 March 2009. extension. URL: <https://wg21.link/cwg794>

[CWG900]

Thomas J. Gritzan. [Lifetime of temporaries in range-based for](https://wg21.link/cwg900). 12 May 2009. extension. URL: <https://wg21.link/cwg900>

[CWG914]

Gabriel Dos Reis. [Value-initialization of array types](https://wg21.link/cwg914). 10 June 2009. extension. URL: <https://wg21.link/cwg914>

[LWG2432]

David Krauss. [initializer_list assignability](https://wg21.link/lwg2432). EWG. URL: <https://wg21.link/lwg2432>

[LWG2813]

Brian Bi. [std::function should not return dangling references](https://wg21.link/lwg2813). EWG. URL: <https://wg21.link/lwg2813>

[P0592R4]

Ville Voutilainen. [To boldly suggest an overall plan for C++23](https://wg21.link/p0592r4). 25 November 2019. URL: <https://wg21.link/p0592r4>

[P1000R4]

Herb Sutter. [C++ IS schedule](https://wg21.link/p1000r4). 14 February 2020. URL: <https://wg21.link/p1000r4>

[P1018r11]

JF Bastien. [C++ Language Evolution status - pandemic edition - 2021/05](https://wg21.link/p1018r11). 1 June 2021. URL: <https://wg21.link/p1018r11>

[P1018r13]

JF Bastien. [C++ Language Evolution status - pandemic edition – 2021/06-2021/08](https://wg21.link/p1018r13). 6 September 2021. URL: <https://wg21.link/p1018r13>

[P1018r9]

JF Bastien. [C++ Language Evolution status - pandemic edition - 2021/01–2021/03](https://wg21.link/p1018r9). 8 March 2021. URL: <https://wg21.link/p1018r9>

[P1169r3]

Barry Revzin, Casey Carter. [static operator\(\)](https://wg21.link/p1169r3). 14 October 2021. URL: <https://wg21.link/p1169r3>

[P1371R3]

Michael Park, Bruno Cardoso Lopes, Sergei Murzin, David Sankel, Dan Sarginson, Bjarne Stroustrup. [Pattern Matching](https://wg21.link/p1371r3). 15 September 2020. URL: <https://wg21.link/p1371r3>

[P1393R0]

D. S. Hollman, Chris Kohlhoff, Bryce Lebach, Jared Hoberock, Gordon Brown, Michał Dominiak. [A General Property Customization Mechanism](https://wg21.link/p1393r0). 13 January 2019. URL: <https://wg21.link/p1393r0>

[P1401R3]

Andrzej Krzemiński. [Narrowing contextual conversions to bool](https://wg21.link/p1401r3). 27 May 2020. URL: <https://wg21.link/p1401r3>

[P1467R4]

David Olsen, Michał Dominiak. [Extended floating-point types and standard names](https://wg21.link/p1467r4). 14 June 2020. URL: <https://wg21.link/p1467r4>

[P1467r8]

David Olsen, Michał Dominiak, Ilya Burylov. [Extended floating-point types and standard names](https://wg21.link/p1467r8). 16 December 2021. URL: <https://wg21.link/p1467r8>

[P1494r2]

S. Davis Herring. [Partial program correctness](https://wg21.link/p1494r2). 13 November 2021. URL: <https://wg21.link/p1494r2>

[P1701R1]

Nathan Sidwell. [Inline Namespaces: Fragility Bites](https://wg21.link/p1701r1). 13 September 2020. URL: <https://wg21.link/p1701r1>

[P1726R3]

Paul E. McKenney, Maged Michael, Jens Mauer, Peter Sewell, Martin Uecker, Hans Boehm, Hubert Tong, Niall Douglas, Will Deacon, Michael Wong, and David Goldblatt. [Pointer lifetime-end zap](https://wg21.link/p1726r3). 21 February 2020. URL: <https://wg21.link/p1726r3>

[P1774r5]

Timur Doumler. [Portable assumptions](https://wg21.link/p1774r5). 16 December 2021. URL: <https://wg21.link/p1774r5>

[P1858R2]

Barry Revzin. [Generalized pack declaration and usage](https://wg21.link/p1858r2). 1 March 2020. URL: <https://wg21.link/p1858r2>

[P1875R0]

Michael Spear, Hans Boehm, Victor Luchangco, Michael Scott, Michael Wong. [Transactional Memory Lite Support in C++](https://wg21.link/p1875r0). 7 October 2019. URL: <https://wg21.link/p1875r0>

[P1938R1]

Barry Revzin, Daveed Vandevoorde, Richard Smith, Andrew Sutton. [if consteval](https://wg21.link/p1938r1). 2 March 2020. URL: <https://wg21.link/p1938r1>

[P1949R4]

Steve Downey, Zach Laine, Tom Honermann, Peter Bindels, Jens Maurer. [C++ Identifier Syntax using Unicode Standard Annex 31](#). 5 June 2020. URL: <https://wg21.link/p1949r4>

[P1949R6]

Steve Downey, Zach Laine, Tom Honermann, Peter Bindels, Jens Maurer. [C++ Identifier Syntax using Unicode Standard Annex 31](#). 15 September 2020. URL: <https://wg21.link/p1949r6>

[P1967R2]

JeanHeyd Meneide. [#embed - a simple, scannable preprocessor-based resource acquisition method](#). 2 March 2020. URL: <https://wg21.link/p1967r2>

[P1999R0]

Ville Voutilainen. [Process proposal: double-check evolutionary material via a Tentatively Ready status](#). 25 November 2019. URL: <https://wg21.link/p1999r0>

[P2011R1]

Barry Revzin, Colby Pike. [A pipeline-rewrite operator](#). 16 April 2020. URL: <https://wg21.link/p2011r1>

[P2013R2]

Ben Craig. [Freestanding Language: Optional ::operator new](#). 14 August 2020. URL: <https://wg21.link/p2013r2>

[P2025R1]

Anton Zhilin. [Guaranteed copy elision for return variables](#). 15 June 2020. URL: <https://wg21.link/p2025r1>

[P2044R0]

Robert Leahy. [Member Templates for Local Classes](#). 12 January 2020. URL: <https://wg21.link/p2044r0>

[P2071r1]

Tom Honermann, Steve Downey, Peter Bindels, Corentin Jabot, R. Martinho Fernandes. [Named universal character escapes](#). 9 November 2021. URL: <https://wg21.link/p2071r1>

[P2139R1]

Alisdair Meredith. [Reviewing Deprecated Facilities of C++20 for C++23](#). 15 June 2020. URL: <https://wg21.link/p2139r1>

[P2139R2]

Alisdair Meredith. [Reviewing Deprecated Facilities of C++20 for C++23](#). 15 July 2020. URL: <https://wg21.link/p2139r2>

[P2145R1]

Bryce Adelstein Lelbach, Titus Winters, Fabio Fracassi, Billy Baker, Nevin Liber, JF Bastien, David Stone, Botond Ballo, Tom Honermann. [Evolving C++ Remotely](#). 15 September 2020. URL: <https://wg21.link/p2145r1>

[P2156R0]

Erich Keane. [Allow Duplicate Attributes](#). 15 April 2020. URL: <https://wg21.link/p2156r0>

[P2173R0]

Daveed Vandevoorde, Inbal Levi, Ville Voutilainen. [Attributes on Lambda-Expressions](#). 15 May 2020. URL: <https://wg21.link/p2173r0>

[P2174R0]

Zhihao Yuan. [Compound Literals](#). 16 May 2020. URL: <https://wg21.link/p2174r0>

[P2186R0]

JF Bastien, Alisdair Meredith. [Removing Garbage Collection Support](#). 12 July 2020. URL: <https://wg21.link/p2186r0>

[P2195R0]

Bryce Adelstein Lelbach. [Electronic Straw Polls](#). 15 September 2020. URL: <https://wg21.link/p2195r0>

[P2201R0]

Jens Maurer. [Mixed string literal concatenation](#). 14 July 2020. URL: <https://wg21.link/p2201r0>

[P2223R0]

Corentin Jabot. [Trimming whitespaces before line splicing](#). 14 September 2020. URL: <https://wg21.link/p2223r0>

[P2266r2]

Arthur O'Dwyer. [Simpler implicit move](#). 20210831. URL: <https://wg21.link/p2266r2>

[P2280r3]

Barry Revzin. [Using unknown references in constant expressions](#). 18 August 2021. URL: <https://wg21.link/p2280r3>

[P2290r2]

Corentin Jabot. [Delimited escape sequences](#). 15 July 2021. URL: <https://wg21.link/p2290r2>

[P2295r5]

Corentin Jabot, Peter Brett. [Support for UTF-8 as a portable source file encoding](#). 16 July 2021. URL: <https://wg21.link/p2295r5>

[P2324r1]

Martin Uecker. [Labels at the end of compound statements \(C compatibility\)](#). 14 October 2021. URL: <https://wg21.link/p2324r1>

[P2327r1]

Paul M. Bendixen, Jens Maurer, Arthur O'Dwyer, Ben Saks. [De-deprecating volatile compound operations](#). 4 October 2021. URL: <https://wg21.link/p2327r1>

[P2348r2]

Corentin Jabot. [Whitespaces Wording Revamp](#). 9 October 2021. URL: <https://wg21.link/p2348r2>

[P2350r2]

Andreas Fertig. [constexpr class](#). 14 October 2021. URL: <https://wg21.link/p2350r2>

[P2362r3]

Peter Brett, Corentin Jabot. [Remove non-encodable wide character literals and multicharacter wide character literals](#). 27 August 2021. URL: <https://wg21.link/p2362r3>

[P2437r0]

Aaron Ballman. [Support for #warning](#). 9 September 2021. URL: <https://wg21.link/p2437r0>

[P2448r0]

Barry Revzin. [Relaxing some constexpr restrictions](#). 14 October 2021. URL: <https://wg21.link/p2448r0>

[P2468r1]

Barry Revzin, Bjarne Stroustrup, Cameron DaCamara, Daveed Vandevoorde, Gabriel Dos Reis, Herb Sutter, Jason Merrill, Jonathan Caves, Richard Smith, Ville Voutilainen. [The Equality Operator You Are Looking For](#). 16 December 2021. URL: <https://wg21.link/p2468r1>